

SKIN LESION CLASSIFICATION USING GNN AND ViT

A CAPSTONE PROJECT REPORT

*Submitted in partial fulfillment of the
requirement for the award of the
Degree of*

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

by

PENDELA BSM VARUN SAI (22BCE7303)

SUKASI NIKHIL KUMAR (22BCE7804)

MEDA SANTHOSH KUMAR (22BCE9811)

PIKKILI RANGA PAVAN (22BCE9949)

Under the Guidance of

DR. RAJASEKHAR BODDU



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237**

DECEMBER 2025

CERTIFICATE

This is to certify that the Capstone Project work titled “**SKIN LESION CLASSIFICATION USING GNN AND ViT**” that is being submitted by **PENDELA BSM VARUN SAI (22BCE7303), SUKASI NIKHIL KUMAR (22BCE7804), MEDA SANTHOSH KUMAR (22BCE9811), PIKKILI RANGA PAVAN (22BCE9949)** is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. RAJASEKHAR BODDU

Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner

External Examiner

Approved by

PROGRAM CHAIR

B. Tech. CSE

DEAN

School Of Computer Science
and Engineering

ACKNOWLEDGEMENTS

We express our heartfelt gratitude to our project guide, Dr. Rajasekhar Boddu, for his expert mentorship, insightful feedback, and constant support during every phase of our capstone project. His guidance has been instrumental in shaping the direction and quality of this work. We also thank VIT-AP University for offering an excellent academic environment and providing the infrastructure required for the smooth execution of this project. We are grateful to all faculty members, peers, and well-wishers who supported us and contributed to our learning experience throughout this journey.

ABSTRACT

Skin cancer is one of the most prevalent and potentially life-threatening diseases around the world, and early detection by automated skin lesion classification can considerably help improve patient outcomes. This paper proposes a hybrid deep learning model combining the Vision Transformer (ViT) and Graph Neural Network (GNN) architectures for robust skin lesion classification using the HAM10000 dataset. The ViT model extracts high-dimensional image representations leveraging self-attention mechanisms to capture global spatial dependencies, while the GNN refines these features in a manner that captures inter-sample relationships based on similarity graphs. The data preprocessing consisted of normalization, augmentation, and stratified sampling in order to handle class imbalance. Experimental results are presented, showing that the proposed ViT-GNN model yields superior accuracy compared to traditional CNN-based approaches, thereby effectively integrating the global feature extraction capability of transformers with the relational reasoning ability of graph networks. This approach points to a promising perspective of transformer-graph hybrid architectures in further advancing computer-aided dermatological diagnosis.

TABLE OF CONTENTS

S.No.	Chapter	Title	Page Number
1.		Acknowledgement	2
2.		Abstract	3
3.		List of Figures and Table	5
4.	1	Introduction	6
	1.1	Objectives	7
	1.2	Background and Literature Survey	7
	1.3	Organization of the Report	9
5.	2	Skin Lesion Classification using GNN and ViT	10
	2.1	Proposed System	10
	2.2	Dataset	10
	2.3	Working Methodology	13
	2.4	System Details	15
	2.4.1	Software	15
	2.4.2	Hardware	17
6.	3	Results and Discussion	18
7.	4	Conclusion & Future Works	23
8.	5	Appendix	24
9.	6	References	37

List of Tables

Table No.	Title	Page No.
1.	Different classes with a number of images in the HAM 10000 dataset	11
2.	Classification report of skin lesions	22

List of Figures

Figure No.	Title	Page No.
1	Overview of GNN-ViT	10
2	Dashboard of the Skin Lesion Classification	15
3	Result Page Showing Lesion Classification	16
4	ROC Curve for ViT-GNN	19
5	Precision–Recall Curve for ViT-GNN	20
6	Confusion Matrix for ViT - GNN Model	21

CHAPTER 1

INTRODUCTION

In recent years, deep learning techniques have shown remarkable ability in dermoscopic image analysis for automated skin lesion classification; Convolutional Neural Networks (CNNs) have been widely used for this task, but they frequently fail to capture long-range spatial dependencies within complex lesion patterns. Skin cancer is still one of the most prevalent and potentially fatal diseases in the world, and early diagnosis is crucial in preventing malignant lesions from progressing to advanced stages.

This limitation has been overcome with the introduction of Vision Transformers (ViTs), which model global relationships across image patches using self-attention mechanisms. Because ViTs can extract comprehensive structural information from dermoscopic images, they have demonstrated strong performance in medical image analysis. Even with this advancement, ViTs might not be able to fully utilize the relational information between various lesion characteristics or image regions. Recognizing these gaps, researchers have started exploring complementary methods like Graph Neural Networks (GNNs).

By depicting image regions or extracted features as connected nodes within a graph, Graph Neural Networks (GNNs) offer a potent framework for modeling relational dependencies. Combining GNNs and ViTs provides a complementary method: GNNs capture structural and neighborhood relationships among feature embeddings, while ViTs extract rich global representations from dermoscopic images. When compared to using either model separately, this hybrid ViT-GNN architecture may increase robustness, decrease misclassification of visually similar lesions, and improve diagnostic performance.

In this report, we propose a combined Vision Transformer and Graph Neural Network model for skin lesion classification using the HAM10000 dataset. This study is motivated by the promising results of ViT-based skin lesion classifiers reported in prior literature [1], and aims to further advance automated skin cancer detection through graph-based relational learning.

1.1 Objectives

The following are the objectives of this project:

- To develop a hybrid deep learning model that combines the strengths of Vision Transformers (ViT) for global feature extraction and Graph Neural Networks (GNN) for relational learning. This aims to achieve accurate and efficient skin lesion classification.
- To use the HAM10000 dataset by performing preprocessing, image normalization, data augmentation, and handling class imbalance. This will ensure robust and unbiased model training.
- To implement effective optimization and regularization techniques such as the AdamW optimizer, cosine annealing for learning rate scheduling, mixup augmentation, and label smoothing. These methods will improve model generalization and stability.
- To design a reliable and clear computer-aided diagnostic system that can assist dermatologists in the early detection and classification of skin cancer. This will improve diagnostic accuracy and decision-making.

1.2 Background and Literature Survey

The initial efforts in skin lesion classification predominantly relied on traditional machine-learning methodologies that necessitated manually crafted features. Ozkan and Koklu showed that algorithms like SVM, ANN, and KNN could sort skin lesions, but they had trouble with changes in shape, color, and texture because they couldn't extract enough features [2]. Hatem also used a KNN-based method to recognize lesions, focusing on simplicity and computational efficiency. However, the method wasn't very strong when dealing with different dermoscopic patterns [3]. These limitations showed how weak handcrafted-feature-based models are and made it clear that we need better ways to learn features.

Deep learning made convolutional neural networks (CNNs) a lot better at analyzing skin images. Yap et al. developed a multimodal deep-learning system that integrated dermoscopic images with patient metadata, showing significant advancements compared to traditional machine learning methods [4]. Seeja and Suresh improved the detection of melanoma even more by combining deep-learning-based segmentation with SVM classification. This cut down on background noise before analyzing lesions [4]. Even though CNNs did well, they still had trouble

getting global contextual information because their receptive fields were too small. This led to the search for models that were more aware of the world around them.

By using self-attention mechanisms that can model long-range dependencies within images, transformer-based architectures filled this gap. According to Flosdorf et al., Vision Transformers (ViTs) outperform CNNs in skin lesion classification, demonstrating their superior capacity to capture global structures and high-level semantic relationships [4]. Kartheepan et al. expanded on this by using Swin Transformer models, whose hierarchical attention design made it possible to handle multi-scale lesion patterns more effectively [6]. Furthermore, Shaik et al. emphasized the significance of attention mechanisms in capturing intricate lesion features by proposing an attention-driven hybrid CNN–BiLSTM model [7]. Together, these pieces showed how attention-driven and transformer-based systems are becoming more and more popular.

The most recent approach to modeling structural relationships in medical images is graph-based learning. In order to help the model learn inter-region dependencies that pixel-based approaches frequently overlook, Alwakid et al. developed a Graph Attention Network (GAT) framework that builds graphs representing lesion regions using CNN-generated node features [8]. The potential advantages of combining global feature extraction (via ViTs) with relational reasoning (via GNNs) are illustrated by insights from the literature, ranging from classical ML to CNNs, attention-based hybrids, and GNNs. This serves as the basis and inspiration for the hybrid ViT-GNN model created in this study.

Collectively, the literature shows a significant trend toward graph-based and attention-driven approaches for classifying skin lesions. Nevertheless, these methods—either transformer-based or graph-based—are usually applied separately. In order to achieve more dependable and clinically significant diagnostic performance, a hybrid framework that combines GNN's relational modeling with ViT's global feature extraction is required.

1.3 Organization of the Report

The remaining chapters of the project report are described as follows:

- Chapter 2 contains the proposed system, methodology, hardware and software details.
- Chapter 3 implementation of the project.
- Chapter 4 discusses the results obtained after the project was implemented.
- Chapter 5 concludes the report.
- Chapter 6 consists of codes.
- Chapter 7 gives references.

CHAPTER 2

SKIN LESION CLASSIFICATION USING GNN AND ViT

This Chapter describes the proposed system, working methodology, software and hardware details.

2.1 Proposed System

The following block diagram (figure 1) shows the overview of this project.

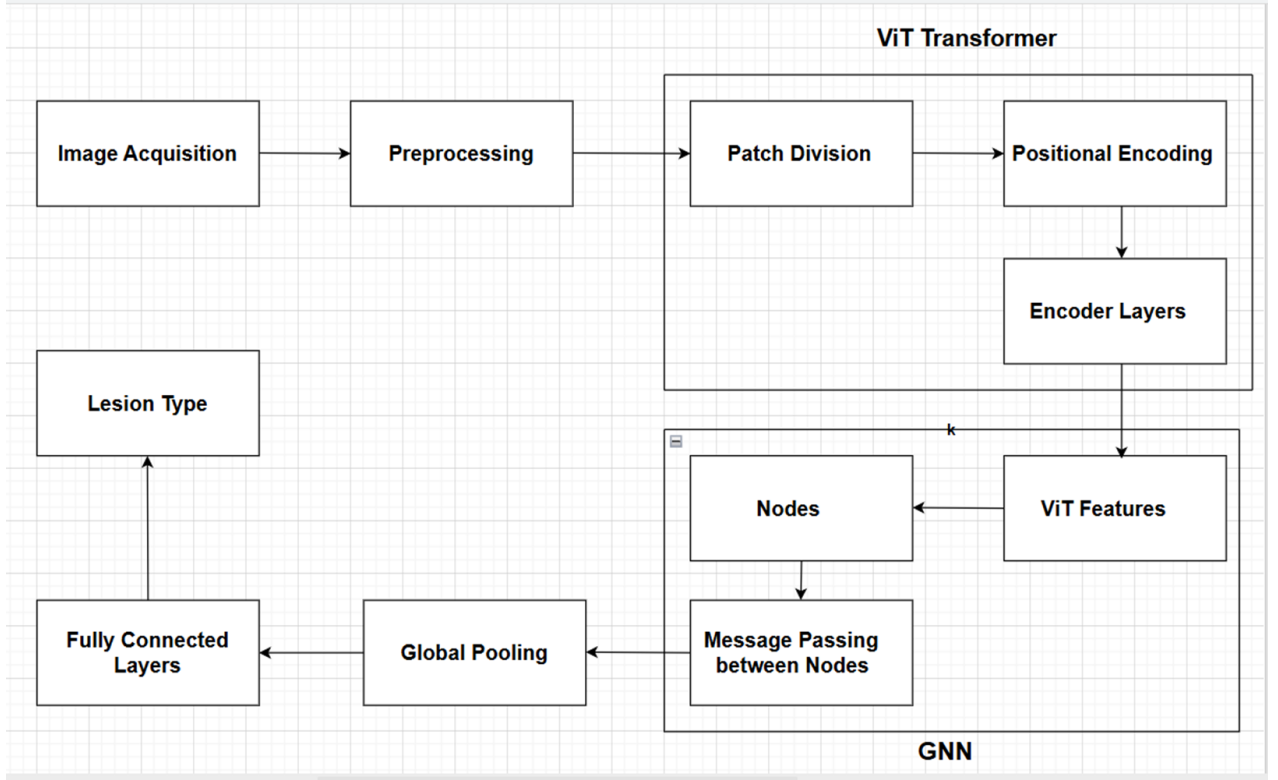


Figure 1 Overview of GNN-ViT

2.2 Dataset

The HAM10000 dataset, which stands for Human Against Machine with 10,000 training images, is a large collection of dermoscopic images. It contains 10,015 high-quality examples of pigmented skin lesions. The classification system includes seven diagnostic categories: Melanocytic nevi (NV), Melanoma (MEL), Benign keratosis-like lesions (BKL), Basal cell carcinoma (BCC), Actinic keratoses and intraepithelial carcinoma (AKIEC), Dermatofibroma (DF), and Vascular lesions (VASC). The images come from different clinical settings, imaging methods, and patient groups. This leads to natural variations in skin tone, lighting, lesion size, color

patterns, and typical dermoscopic artifacts, like hair or ruler marks. Each image is linked to a diagnosis confirmed by experts, based on histopathology, clinical follow-up, or agreement among specialists. This provides a reliable foundation for research and the development of models used in automated skin cancer detection.

TABLE I

Different classes with a number of images in the HAM 10000 dataset

Classes	Abbreviation	No. of images
Actinic keratoses and intraepithelial carcinoma	AKIEC	327
Basal Cell Carcinoma	BCC	514
Benign Keratosis-like lesions	BKL	1099
Dermatofibroma	DF	115
Melanoma	MEL	1113
Melanocytic nevi	NV	6705
Vascular lesions	VASC	142

A significant feature of the HAM10000 dataset is its highly imbalanced class distribution. Common lesion types, such as NV, make up a large part of the dataset, while less common categories, like DF and VASC, have very few examples. This imbalance mirrors real-world dermatology, creating methodological challenges. To achieve balanced model learning, techniques

like data augmentation, weighted loss functions, or sampling strategies are necessary. Despite these limitations, the dataset's wide variety and realistic clinical setting make it a valuable reference for evaluating advanced deep learning systems. This approach is particularly well-suited for hybrid architectures, including Vision Transformers and Graph Neural Networks, thereby facilitating rigorous experimentation in feature learning, cross-lesion generalization, and robust melanoma detection; consequently, it supports the advancement of precise and dependable computer-aided diagnostic tools.

Data Preprocessing and Augmentation

The preprocessing steps described are essential to establish a standard in the dermoscopic images from HAM10000 so that they are suitable for deep learning model training. Given that it is a dataset from different clinical sources, the raw images vary regarding resolution, illumination, color tone, and the presence of certain artifacts such as vignetting, ruler marks, or body hair. To prepare the data, each image is first resized into a consistent spatial dimension- typically 224×224 pixels for Vision Transformer inputs-and normalized using the ImageNet dataset mean and standard deviation to match the pre-trained weights. At the preprocessing stage, it is also possible to conduct color normalization in order to reduce differences in illumination, convert these images to tensors, and optionally remove hairs or suppress other artifacts, if required for better clinical clarity. Labels are encoded as numerical identifiers, and the dataset is split into training, validation, and testing sets with stratified sampling to maintain class proportion.

In order to address the inherent class imbalance and improve the model's capacity to generalize across a variety of lesion appearances, data augmentation is crucial. A range of transformations, such as random horizontal and vertical flips, rotations, cropping, zooming, color jittering, Gaussian noise, and contrast adjustments that mimic real-world variations observed in dermoscopic images, are applied during training to artificially expand the dataset. To help the model learn stable and invariant representations, advanced policies like RandAugment and AutoAugment further introduce controlled distortion. By offering a variety of patch-level patterns, these augmentation techniques support the Graph Neural Network and aid the Vision Transformer in learning richer contextual features. When combined, preprocessing and augmentation greatly increase model robustness, decrease overfitting, and boost skin lesion classification accuracy.

2.3 Working Methodology

This system is a combined deep learning architecture, integrating Vision Transformer (ViT) and Graph Neural Network (GNN) models, to improve the accuracy of classifying dermoscopic skin lesion images. The goal is to enhance feature extraction and classification performance, particularly for challenging melanoma cases, by combining the global self-attention mechanism of Vision Transformers (ViTs) with the relational feature learning capabilities of Graph Neural Networks (GNNs). The proposed method first extracts global image embeddings using patches and a Vision Transformer (ViT). Then, it models the relationships between these patches with a Graph Neural Network (GNN) to capture the structural dependencies within the lesion image. After the graph-level representation is created, it's used as input for a classifier that categorizes skin lesions.

Vision Transformer (ViT) Mechanism :

The Vision Transformer (ViT) forms the initial stage of the model and is responsible for extracting high-level global information from dermoscopic pictures. Traditional convolutional neural networks rely primarily on localized receptive fields, which might impede their capacity to capture long-range relationships prevalent in medical imagery. ViT tackles this restriction by totally replacing convolutions with a transformer-based architecture originally built for natural language processing. Initially, the input image is split up into a series of fixed-size patches, which are then flattened and projected into a latent vector space. These patch embeddings are used similarly to word tokens in NLP, enabling the transformer to process the entire image as a sequence.

To preserve spatial information lost during flattening, ViT adds positional encodings into each patch embedding. These encodings allow the transformer to preserve awareness of the relative position of patches within the original image. The sequence of embedded patches is processed via various transformer encoder levels consisting of multi-head self-attention devices. Through self-attention, ViT is able to represent global interactions by calculating the relevance of every patch relative to all other patches in the image. This technique is particularly effective in skin lesion analysis, as it captures fine-grained texture patterns, irregular color fluctuations, asymmetric boundaries, and bigger structural clues that are crucial for diagnosing malignancies. A strong, high-

dimensional feature representation of the image that encodes its most discriminative characteristics is the end result of the ViT component.

Graph Neural Network (GNN) Mechanism :

Graph Neural Network (GNN) is used in the model's second stage to improve classification by introducing relational reasoning into the feature space. GNNs function by modeling the relationships and structural dependencies between samples, whereas ViT concentrates on extracting features at the individual picture level. After the ViT develops a feature embedding for each image, these embeddings are processed as nodes inside a graph. To link each node with other nodes that have comparable feature traits, edges are built using a predetermined similarity measure, typically k-nearest neighbor relationships.

The GNN carries out iterative message-passing operations after the graph structure is created. In each layer, a node updates its feature representation by collecting information from its neighbors. This aggregation stage helps GNN to collect higher-order relationships across the dataset, enabling it to learn patterns that go beyond the individual image. For instance, lesions that share comparable structural or visual properties become strongly connected in the graph, allowing the model to convey useful information across related samples. This is especially advantageous in medical picture classification if class imbalance exists, as the GNN helps reinforce class-specific patterns by utilizing neighborhood context. The conclusion of GNN is an improved collection of embeddings that contain both the intrinsic features recovered by ViT and the structural links acquired through graph.

A hybrid architecture that combines the advantages of relational learning and global feature extraction is produced by combining ViT and GNN. ViT provides a rich and holistic representation of each skin lesion image, while the GNN extends this representation by grouping embeddings in a structured graph space and learning inter-sample correlations. A classification layer receives the final node embeddings generated by the GNN and outputs the anticipated skin lesion category. In datasets like HAM10000, where class variability, visual overlap, and inter-class similarity might present difficulties for standalone deep learning models, this integrated strategy allows the model to attain higher robustness.

2.4 System Details

This section describes the software and hardware details of the system:

2.4.1 Software Details

Language: Python 3.8+

Frameworks: PyTorch, TorchVision, Flask

Libraries: PyTorch Geometric, scikit-learn, NumPy, Pandas

Tools: Google colab

This project was implemented using Python 3.8+ as the primary programming language, leveraging several powerful deep learning and data processing frameworks. PyTorch and TorchVision were used for building and training deep neural network models, while PyTorch Geometric enabled the integration of Graph Neural Network (GNN) components for advanced relational learning. Additional libraries such as scikit-learn, NumPy, and Pandas were employed for data preprocessing, feature handling, and performance evaluation. For deployment and API creation, a lightweight backend was developed using the Flask framework. The entire workflow, including model development and experimentation, was executed using Google Colab, providing GPU-accelerated training and an efficient cloud-based development environment.

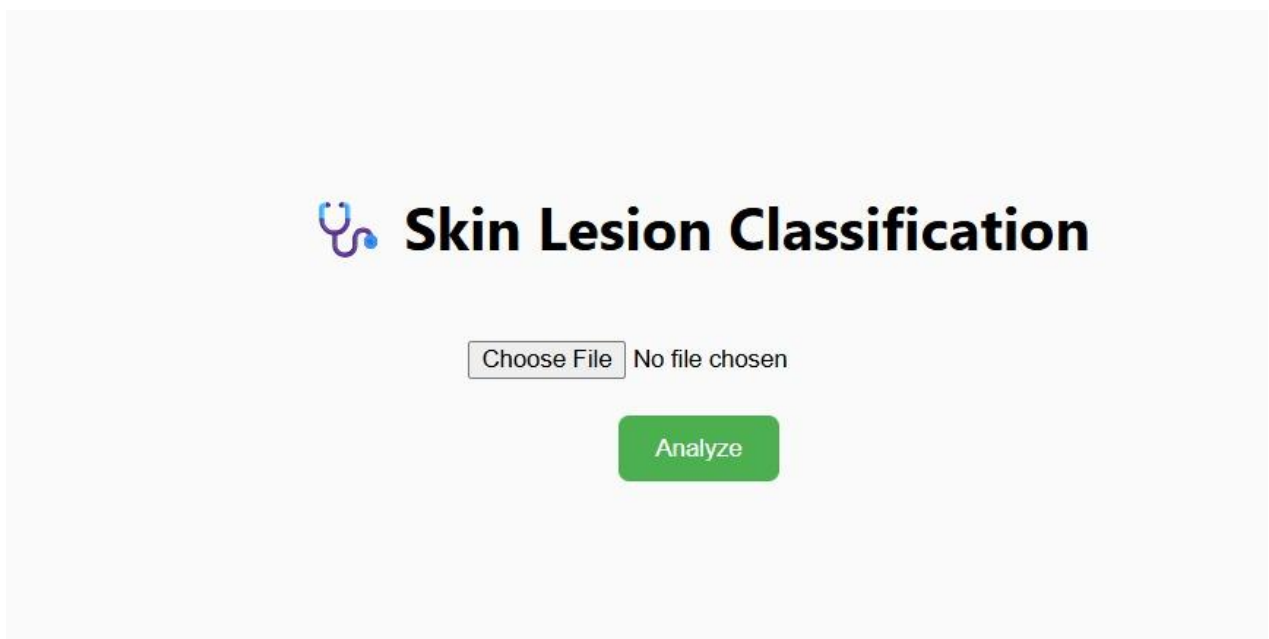


Fig.2. Dashboard of the Skin Lesion Classification

The Flask micro-framework, which offers a straightforward yet effective backend structure for implementing machine learning models online, was used in the development of the skin lesion classification website. Flask manages routing, file uploading, and interaction between the user interface and the trained ViT–GNN model. When a user selects an image, Flask gets the file, preprocesses it, and transmits it to the model for prediction. After that, the outcome is returned and shown on an HTML page that has been dynamically rendered. The front-end interface is designed using HTML, CSS, and Bootstrap, delivering a clean and responsive design. Real-time skin lesion analysis through an intuitive web application is made possible by Flask's lightweight architecture, which makes the system quick, effective, and simple to combine with deep learning workflows.

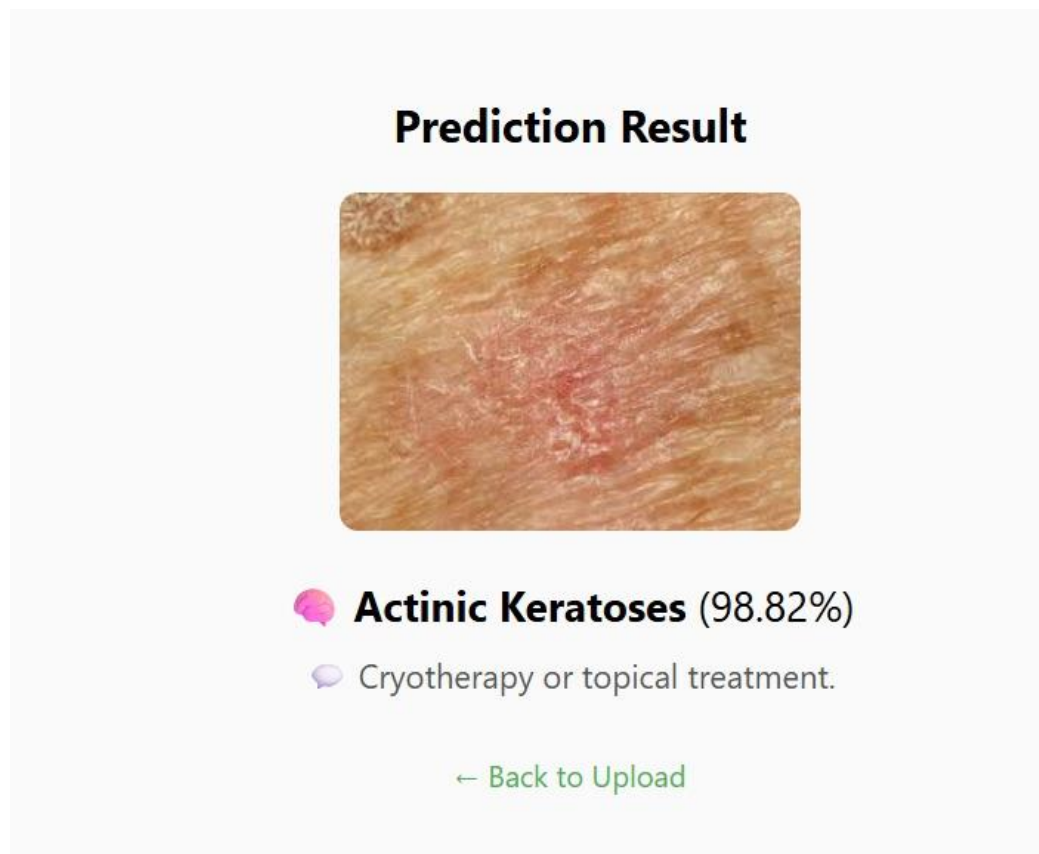


Fig.3. Result Page Showing Lesion Classification Output

2.4.2 Hardware Details

The hardware setup for this project consisted of a local development environment powered by an Intel Core i7 processor with 16 GB RAM, which was used for all stages of implementation, including data preprocessing, model training, and evaluation. The entire deep learning workflow—covering Vision Transformer (ViT) processing, Graph Neural Network (GNN) computations, and Flask-based backend deployment—was executed on this CPU-only system without the use of any dedicated GPU hardware. Despite the computational intensity of the models, optimizations such as batch size tuning and efficient data loading enabled smooth experimentation and reliable performance on the available i7-based machine. This setup demonstrates the feasibility of developing and testing the hybrid deep learning architecture even in a standard CPU environment.

CHAPTER 3

RESULTS AND DISCUSSIONS

The effectiveness of the proposed ViT-GNN hybrid architecture was thoroughly evaluated using the HAM10000 dataset, and the results demonstrate strong predictive performance across all seven skin lesion categories. The model achieved a validation accuracy of 95.47% and a test accuracy of 94.08%, indicating that the learned feature representations generalize well from training to unseen patient data. Such consistency reflects the ability of the integrated ViT module to capture global contextual information, while the GNN component further refines classification by modeling inter-sample relational structures within the feature space.

Moreover, the stable margin between validation and test accuracy suggests that the model avoids overfitting despite the natural imbalance and visual variability present in dermoscopic images. The high separability across classes, supported by ROC and PR curve patterns, further confirms that the combined architecture learns discriminative patterns even for visually similar lesion types. This strong performance highlights the advantage of merging spatial attention mechanisms with graph-based reasoning to improve robustness. Overall, the results demonstrate that the ViT-GNN framework not only enhances classification reliability but also brings a deeper structural understanding of the underlying data distribution, which is crucial for real-world clinical applicability.

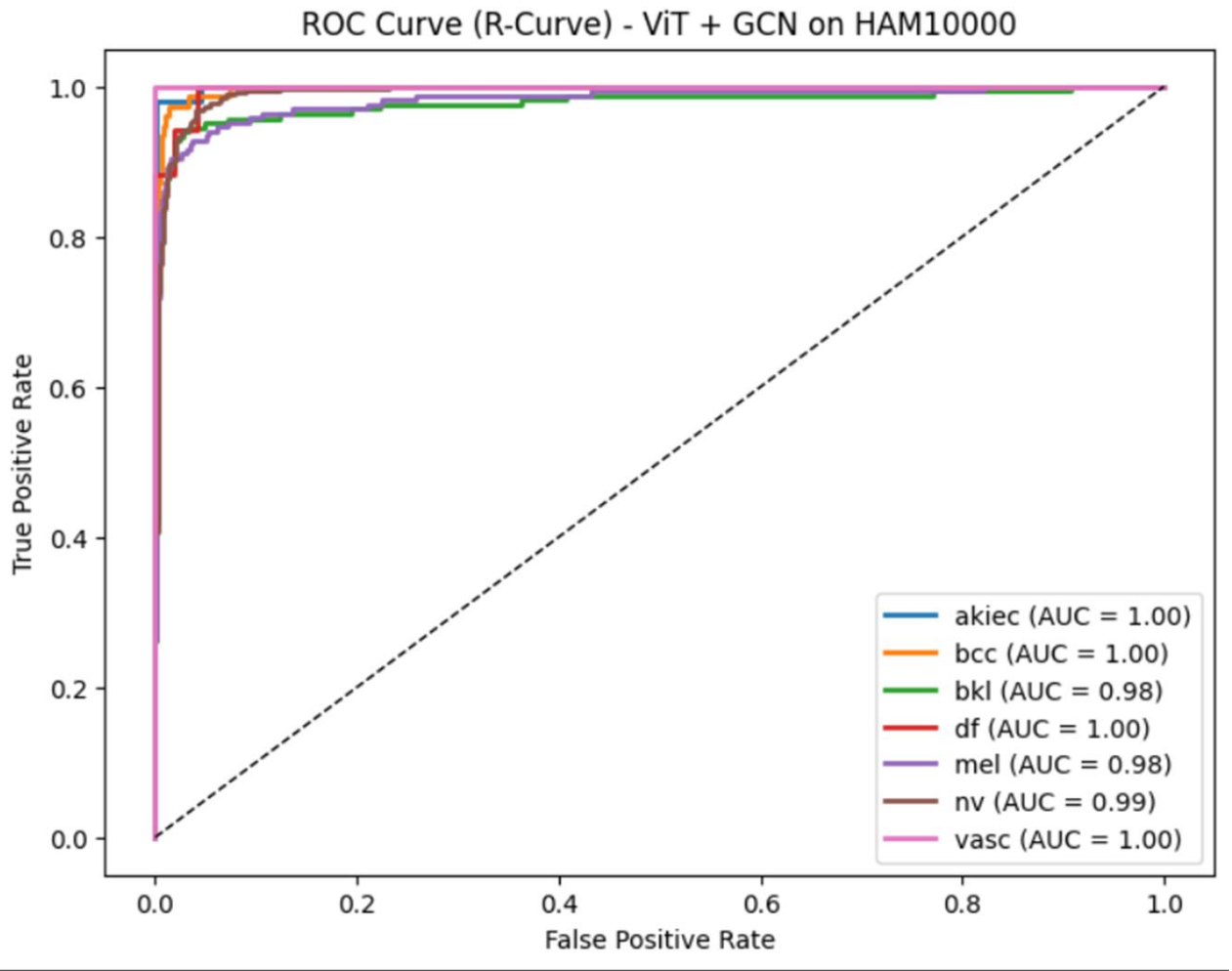


Fig.4. ROC Curve for ViT - GNN

Furthermore, performance under class imbalance was examined using Precision–Recall (PR) curves, which are crucial because the HAM10000 dataset has a highly unequal distribution of lesion types. When the cost of false negatives is high, as in melanoma detection, PR curves offer a deeper understanding of the model's behavior. Even as recall rises, the suggested model consistently shows high precision values, which is essential for lowering the number of malignant cases that are missed. This performance demonstrates that even in the presence of extremely similar inter-class patterns, discriminative clarity can be maintained by combining ViT features with graph-based refinement.

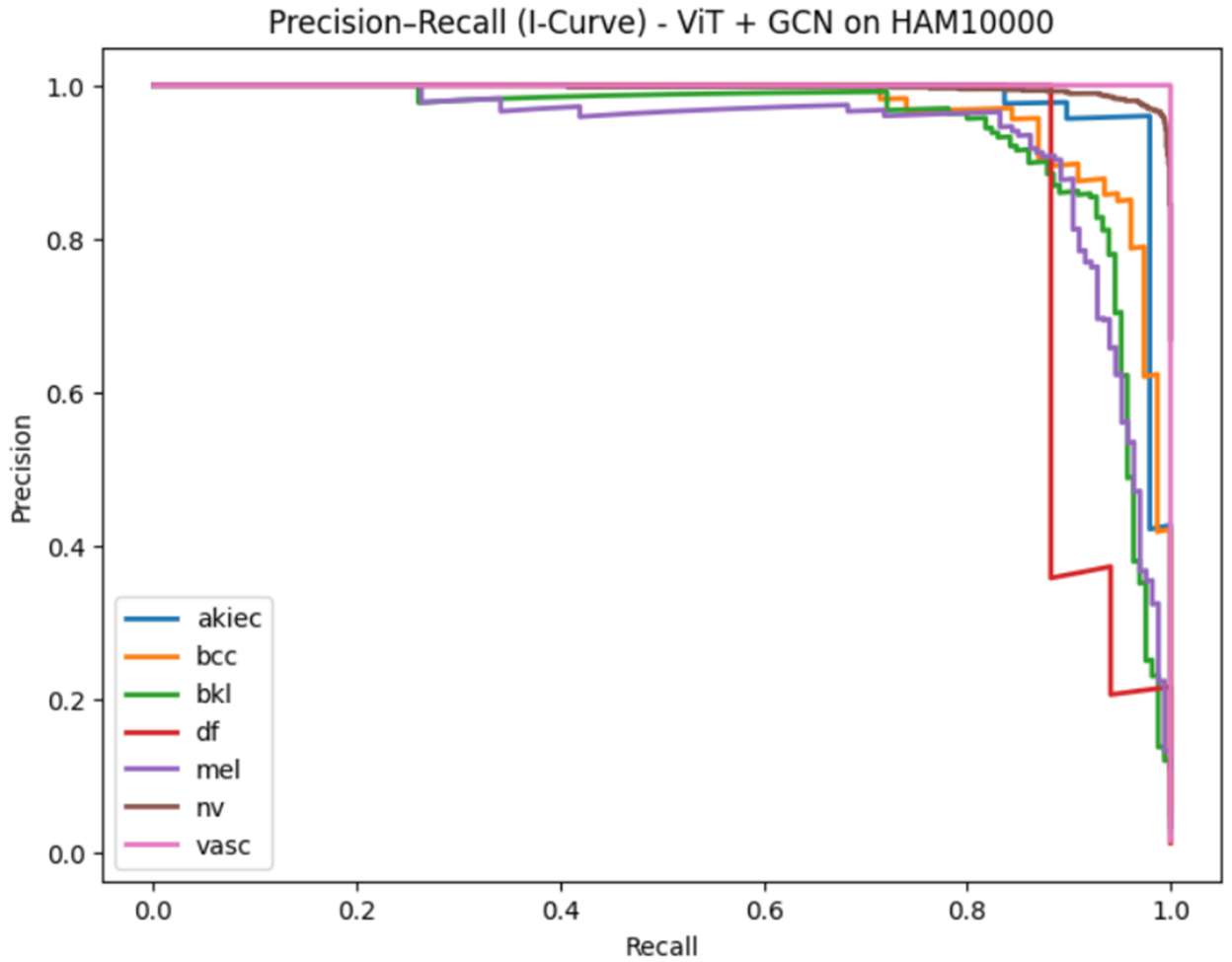


Fig.5. Precision-Recall Curve for ViT - GNN

To evaluate the class-wise predictive behavior, a thorough confusion matrix was also examined. With lesion types like NV achieving 964 correct predictions, MEL achieving 151 correct predictions, and BKL achieving 144 correct predictions, the model exhibits strong diagonal accuracy, demonstrating the architecture's ability to distinguish between both commonly occurring and clinically challenging classes. The majority of misclassifications happen between visually similar categories, like MEL versus NV and BKL versus BCC, where even skilled dermatologists are unable to distinguish subtle dermoscopic differences. These small misunderstandings highlight the inherent difficulty of skin lesion analysis and support the need for hybrid models that integrate relational learning and global reasoning.

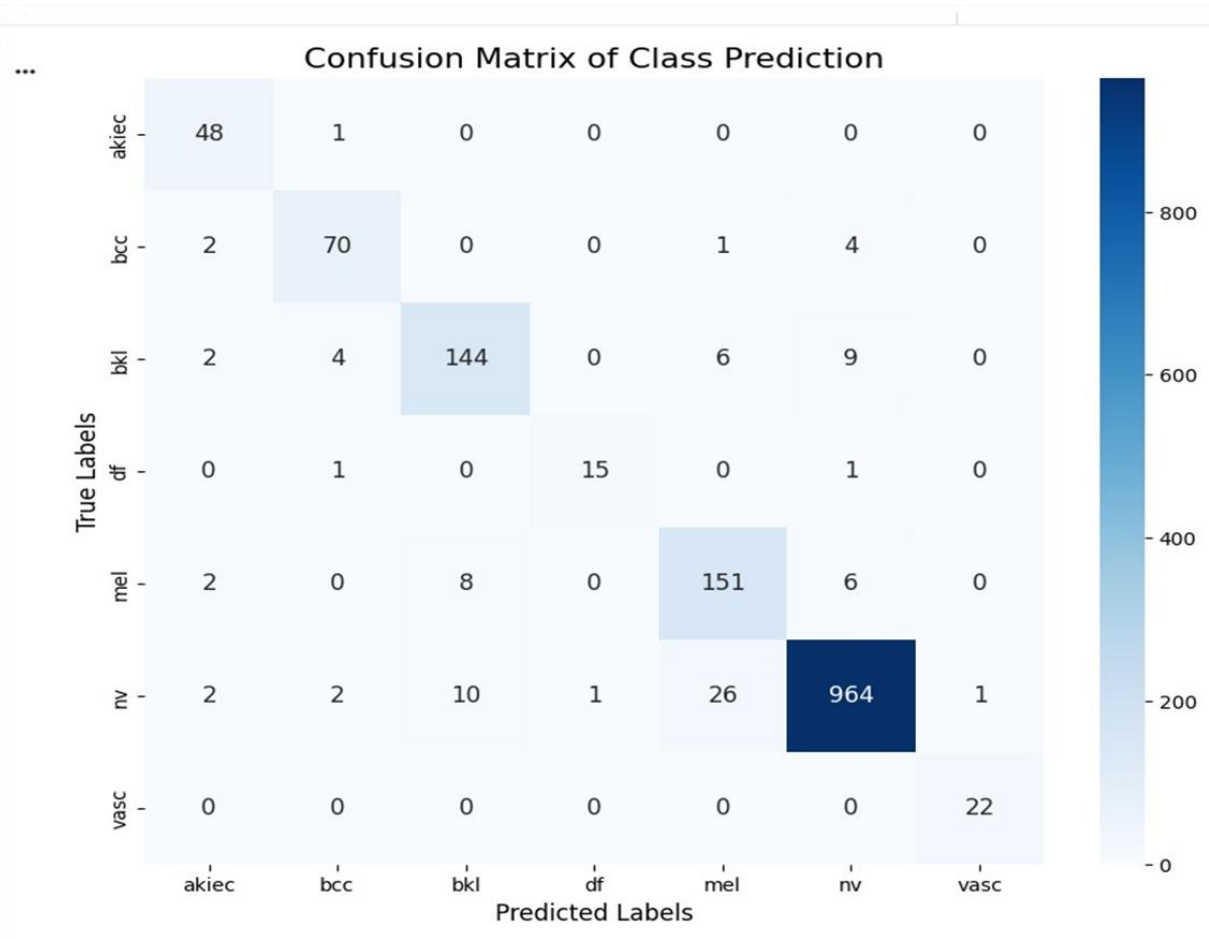


Fig.6. Confusion Matrix for ViT - GNN Model

TABLE II**Classification report of skin lesions**

Classes	Precision	Recall	F1-score
AKIEC	0.86	0.98	0.91
BCC	0.90	0.91	0.90
BKL	0.89	0.87	0.88
DF	0.94	0.88	0.91
MEL	0.82	0.90	0.86
NV	0.98	0.96	0.97
VASC	0.96	1.00	0.98

Overall, it is evident from the analysis of accuracy, AUC, PR curves, and confusion matrix results that the suggested ViT-GNN architecture produces very stable, dependable, and broadly applicable performance. The system overcomes the drawbacks of conventional CNN-based techniques by combining Vision Transformers, which are intended to capture global, long-range dependencies, with Graph Neural Networks, which can model local structural coherence. The model is therefore a promising solution for practical dermatological diagnostic applications since it not only achieves strong class separation and resilience against dataset imbalance, but also performs exceptionally well in classification accuracy.

CHAPTER 4

CONCLUSION AND FUTURE WORK

We introduced a hybrid Vision Transformer–Graph Neural Network (ViT-GNN) model designed to improve the accuracy and reliability of skin lesion classification. By combining the global attention capabilities of ViT with the structural learning strengths of GNNs, our approach brings together the best of both worlds—rich image-level understanding and meaningful relational reasoning between samples. This synergy allows the model to capture subtle variations in skin lesions that are often challenging even for traditional deep learning models.

The experimental results on the HAM10000 dataset demonstrate the strength of our approach. Achieving a validation accuracy of 95.47% and a test accuracy of 94.08%, the model shows strong generalization from training data to real-world, unseen cases. These results highlight not only the ability of ViT to extract comprehensive global features but also the critical role of GNN in refining decision boundaries based on relationships within the feature space. The consistent performance across all lesion categories further reinforces the potential of this hybrid architecture for practical clinical use.

Overall, the proposed ViT-GNN framework represents an important step forward in automated dermatological diagnosis. Its high accuracy, strong generalization, and adaptability make it a promising candidate for real-time, AI-assisted skin cancer screening tools. Moving forward, the model can be extended by integrating multimodal patient data, exploring self-supervised pretraining, or optimizing the architecture for deployment on handheld medical devices. Such developments can help bring advanced diagnostic support closer to clinicians and patients, ultimately contributing to earlier detection and improved treatment outcomes.

CHAPTER 5

APPENDIX

Code

1. Installing Dependencies and Configure Kaggle API Access

```
!pip install torch-scatter torch-sparse torch-cluster torch-spline-conv torch-geometric kaggle -q

import os

from google.colab import files

KAGGLE_PATH = os.path.expanduser("~/kaggle/kaggle.json")

if not os.path.exists(KAGGLE_PATH):

    print("Kaggle API token not found. Please upload kaggle.json:")

    uploaded = files.upload()

    if 'kaggle.json' not in uploaded:

        raise Exception("You must upload kaggle.json to proceed!")

    os.makedirs(os.path.dirname(KAGGLE_PATH), exist_ok=True)

    with open(KAGGLE_PATH, 'wb') as f:

        f.write(uploaded['kaggle.json'])

    os.chmod(KAGGLE_PATH, 0o600)

    print("Kaggle API token uploaded and permissions set.")

else:

    print("Kaggle API token already exists.")
```

2. Import Libraries, Initialize Configuration, and Set Global Hyperparameters

```
import os, random, glob, shutil, numpy as np

import torch, torch.nn as nn
```

```

import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader, random_split, WeightedRandomSampler

from torchvision import transforms

from torchvision.models.vision_transformer import vit_b_16, ViT_B_16_Weights

from torchvision.transforms.autoaugment import RandAugment

from PIL import Image

import pandas as pd

from sklearn.preprocessing import LabelEncoder

from torch_geometric.nn import GCNConv

SEED = 42

torch.manual_seed(SEED)

np.random.seed(SEED)

random.seed(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

DATA_DIR = "/content/HAM10000"

IMG_DIR1 = os.path.join(DATA_DIR, "HAM10000_images_part_1")

IMG_DIR2 = os.path.join(DATA_DIR, "HAM10000_images_part_2")

META_CSV = os.path.join(DATA_DIR, "HAM10000_metadata.csv")

OUTPUT_SIZE = 224

BATCH_SIZE = 32

NUM_EPOCHS = 300

LR = 5e-5 # lower for ViT fine-tuning

```

3. Data Augmentation

```

images_dir = os.path.join(os.getcwd(), "HAM10000_images")

os.makedirs(images_dir, exist_ok=True)

```

```

for d in [IMG_DIR1, IMG_DIR2]:

    for jpg in glob.glob(os.path.join(d, "*.jpg")):

        dst = os.path.join(images_dir, os.path.basename(jpg))

        if not os.path.exists(dst):

            shutil.copy2(jpg, dst)

meta = pd.read_csv(META_CSV)

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

meta['label'] = le.fit_transform(meta['dx'])

label_map = {i: c for i, c in enumerate(le.classes_)}

print(" Classes:", label_map)

from torchvision import transforms

from torch.utils.data import Dataset

from PIL import Image

import os

train_tfms = transforms.Compose([

    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),

    transforms.RandomHorizontalFlip(),

    transforms.RandomVerticalFlip(),

    transforms.RandomRotation(25),

    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3),

    transforms.RandomApply([transforms.GaussianBlur(3)], p=0.3), # new

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406],

                          std=[0.229, 0.224, 0.225]),

```

```

        transforms.RandomErasing(p=0.25, scale=(0.02, 0.2))
    ])

val_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
    ])

class SkinDataset(Dataset):
    def __init__(self, df, img_dir, transform=None):
        self.df = df.reset_index(drop=True)
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        img_path = os.path.join(self.img_dir, row['image_id'] + ".jpg")
        img = Image.open(img_path).convert("RGB")

        if self.transform:
            img = self.transform(img)

        label = int(row['label'])

        return img, label

```

4. Training, Testing, Validation Split

```

from torch.utils.data import DataLoader, WeightedRandomSampler

import numpy as np

from sklearn.model_selection import train_test_split

train_df, temp_df = train_test_split(meta, test_size=0.3, stratify=meta['label'],
random_state=SEED)

val_df, test_df = train_test_split(temp_df, test_size=0.5, stratify=temp_df['label'],
random_state=SEED)

print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}")

train_ds = SkinDataset(train_df, images_dir, train_tfms)

val_ds = SkinDataset(val_df, images_dir, val_tfms)

test_ds = SkinDataset(test_df, images_dir, val_tfms)

class_counts = train_df['label'].value_counts().to_dict()

num_samples = len(train_df)

weights = [1.0 / class_counts[label] for label in train_df['label']]

sampler = WeightedRandomSampler(weights, num_samples=num_samples, replacement=True)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sampler,
num_workers=2, pin_memory=True)

val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2,
pin_memory=True)

test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2,
pin_memory=True)

print("DataLoaders ready.")

```

5. ViT-GNN Model

```

import torch

import torch.nn as nn

import torch.nn.functional as F

```

```

from torchvision.models.vision_transformer import vit_b_16, ViT_B_16_Weights

from torch_geometric.nn import GCNConv

class ViT_GCN(nn.Module):

    def __init__(self, num_classes):

        super().__init__()

        self.vit = vit_b_16(weights=ViT_B_16_Weights.DEFAULT)

        self.vit.heads = nn.Identity()

        self.vit.requires_grad_(False)

        self.vit_proj = nn.Sequential(

            nn.Linear(768, 512),

            nn.BatchNorm1d(512),

            nn.ReLU(inplace=True),

            nn.Dropout(0.4),

            nn.Linear(512, 256)

        )

        self.gcn1 = GCNConv(256, 256)

        self.gcn2 = GCNConv(256, 256)

        self.alpha = nn.Parameter(torch.tensor(0.6))

        self.norm = nn.LayerNorm(256)

        self.dropout = nn.Dropout(0.5)

        self.fc = nn.Sequential(

            nn.Linear(256, 128),

            nn.ReLU(inplace=True),

            nn.Dropout(0.4),

            nn.Linear(128, num_classes)

```

```

    )

def forward(self, img):
    vit_features = self.vit(img)

    vit_features = self.vit_proj(vit_features)

    with torch.no_grad():
        sim = torch.cdist(vit_features, vit_features, p=2)

        threshold = torch.quantile(sim, 0.25)

        edge_index = (sim < threshold).nonzero().T

    gcn_features = F.relu(self.gcn1(vit_features, edge_index))

    gcn_features = F.dropout(gcn_features, p=0.3, training=self.training)

    gcn_features = F.relu(self.gcn2(gcn_features, edge_index))

    fused = self.alpha * vit_features + (1 - self.alpha) * gcn_features

    fused = self.norm(fused)

    fused = self.dropout(fused)

    out = self.fc(fused)

    return out

num_classes = len(label_map)

model = ViT_GCN(num_classes).to(DEVICE)

print("model initialized:", model.__class__.__name__)

```

6. Training Utilities(Mixup,Cutmix,Scheduler)

```

import torch

import torch.nn as nn

import numpy as np

from torch.optim import AdamW

```

```

from torch.optim.lr_scheduler import SequentialLR, LinearLR, CosineAnnealingLR

criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

vit_params = []

new_params = []

for name, param in model.named_parameters():
    if "vit" in name:
        vit_params.append(param)
    else:
        new_params.append(param)

optimizer = AdamW([
    {"params": vit_params, "lr": 1e-5},
    {"params": new_params, "lr": 3e-5},
], weight_decay=1e-4)

NUM_EPOCHS = 400

warmup = LinearLR(optimizer, start_factor=0.1, total_iters=5)

main_sched = CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS - 5)

scheduler = SequentialLR(
    optimizer,
    schedulers=[warmup, main_sched],
    milestones=[5]
)

def mixup_data(x, y, alpha=0.4):
    """Applies mixup on input and targets."""
    if alpha <= 0:
        return x, y, y, 1.0

```

```

lam = np.random.beta(alpha, alpha)

index = torch.randperm(x.size(0)).to(x.device)

mixed_x = lam * x + (1 - lam) * x[index, :]

y_a, y_b = y, y[index]

return mixed_x, y_a, y_b, lam

def mixup_criterion(criterion, pred, y_a, y_b, lam):

    return lam * criterion(pred, y_a) + (1 - lam) * criterion(pred, y_b)

print("Optimizer, scheduler, and mixup ready.")

```

7. Training loop with Best Model saving

```

from google.colab import drive

drive.mount('/content/my_drive', force_remount=True)

import os

import torch

import random

import torch.nn as nn

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5, weight_decay=1e-4)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS)

save_path = "/content/my_drive/MyDrive/vit_gcn_ham10000_best.pt"

checkpoint_path = "/content/my_drive/MyDrive/vit_gcn_checkpoint.pth"

best_acc = 0.0

start_epoch = 1

if os.path.exists(checkpoint_path):

    print("Loading checkpoint from Google Drive...")

```

```

checkpoint = torch.load(checkpoint_path, map_location=DEVICE)

missing, unexpected = model.load_state_dict(checkpoint["model_state"], strict=False)

print(f"Model loaded. Missing keys: {missing}, Unexpected keys: {unexpected}")

try:

    optimizer.load_state_dict(checkpoint["optimizer_state"])

    scheduler.load_state_dict(checkpoint["scheduler_state"])

    print("Optimizer and scheduler loaded.")

except Exception as e:

    print(f"Optimizer/Scheduler mismatch: {e}")

    print("Reinitializing optimizer and scheduler from scratch.")

best_acc = checkpoint.get("best_acc", 0)

start_epoch = checkpoint.get("epoch", 0) + 1

print(f"Resumed from epoch {start_epoch} | Best Val Acc = {best_acc:.4f}")

else:

    print(" No checkpoint found, starting fresh training...")

    start_epoch = 0

    best_acc = 0

def run_epoch(dl, train=True):

    model.train() if train else model.eval()

    total_loss, total_correct, total = 0, 0, 0

    if train and torch.cuda.is_available():

        scaler = torch.amp.GradScaler()

    for x, y in dl:

        x, y = x.to(DEVICE), y.to(DEVICE)

        if train:

```

```

optimizer.zero_grad()

with torch.amp.autocast(device_type='cuda' if torch.cuda.is_available() else 'cpu'):

    out = model(x)

    loss = criterion(out, y)

if torch.cuda.is_available():

    scaler.scale(loss).backward()

    scaler.unscale_(optimizer)

    torch.nn.utils.clip_grad_norm_(model.parameters(), 2.0)

    scaler.step(optimizer)

    scaler.update()

else:

    loss.backward()

    torch.nn.utils.clip_grad_norm_(model.parameters(), 2.0)

    optimizer.step()

scheduler.step()

else:

    with torch.no_grad():

        with torch.amp.autocast(device_type='cuda' if torch.cuda.is_available() else 'cpu'):

            out = model(x)

            loss = criterion(out, y)

pred = out.argmax(1)

total += y.size(0)

total_loss += loss.item() * y.size(0)

total_correct += (pred == y).sum().item()

```

```

    return total_loss / total, total_correct / total

for epoch in range(start_epoch, NUM_EPOCHS + 1):

    if epoch == 60:

        for param in model.vit.parameters():

            param.requires_grad = True

        optimizer = torch.optim.AdamW([

            {"params": model.vit.parameters(), "lr": 2e-5},

            {"params": model.gcn1.parameters(), "lr": 3e-5},

            {"params": model.gcn2.parameters(), "lr": 3e-5},

            {"params": model.fc.parameters(), "lr": 3e-5},

        ], weight_decay=1e-4)

        scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=NUM_EPOCHS - epoch)

        print("Unfroze ViT backbone for fine-tuning with layer-wise LR!")

        tr_loss, tr_acc = run_epoch(train_loader, True)

        va_loss, va_acc = run_epoch(val_loader, False)

        print(f'Epoch {epoch}/{NUM_EPOCHS} | Train Acc: {tr_acc:.4f} | Val Acc: {va_acc:.4f}')

        torch.save({

            "epoch": epoch,

            "model_state": model.state_dict(),

            "optimizer_state": optimizer.state_dict(),

            "scheduler_state": scheduler.state_dict(),

            "best_acc": best_acc

        }, checkpoint_path)

        print(f'Checkpoint saved at epoch {epoch} → {checkpoint_path}')

```

```

if va_acc > best_acc:

    best_acc = va_acc

    torch.save({"model": model.state_dict(), "classes": label_map}, save_path)

    print(f"Best model updated (Val Acc={best_acc:.4f}) → {save_path}")

```

8. Final Evaluation on Testing data

```

checkpoint = torch.load(save_path, map_location=DEVICE)

if "model" in checkpoint:

    model.load_state_dict(checkpoint["model"])

else:

    model.load_state_dict(checkpoint)

model.to(DEVICE)

model.eval()

test_loss, test_acc = run_epoch(test_loader, train=False)

print(f"Final Test Accuracy: {test_acc*100:.2f}%")

```

CHAPTER 6

REFERENCES

- [1] C. Flosdorf, J. Engelker, I. Keller, and N. Mohr, “Skin Cancer Detection utilizing Deep Learning: Classification of Skin Lesion Images using a Vision Transformer,” *arXiv preprint*, arXiv:2407.18554, Jul. 2024.
- [2] I. A. Ozkan and M. Koklu, “Skin Lesion Classification using Machine Learning Algorithms,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 5, no. 4, pp. 285–289, Dec. 2017, doi: 10.18201/ijisae.2017534420.
- [3] M. Q. Hatem, “Skin lesion classification system using a K-nearest neighbour algorithm,” *Visual Computing for Industry, Biomedicine and Art*, vol. 5, no. 7, Mar. 2022, doi: 10.1186/s42492-022-00103-6.
- [4] J. Yap, W. Yolland, and P. Tschandl, “Multimodal skin lesion classification using deep learning,” *Experimental Dermatology*, vol. 27, no. 8, pp. 1–8, Sep. 2018, doi: 10.1111/exd.13777.
- [5] R. D. Seeja and A. Suresh, “Deep Learning Based Skin Lesion Segmentation and Classification of Melanoma Using Support Vector Machine (SVM),” *Asian Pacific Journal of Cancer Prevention*, vol. 20, no. 5, pp. 1555–1561, May 2019, doi: 10.31557/APJCP.2019.20.5.1555.
- [6] M. Kartheepan, R. Rajalakshmi, S. Kandasamy, and S. M. Hari Karthik, “Skin Lesion Analysis using Swin Algorithms,” *International Journal of Creative Research Thoughts (IJCRT)*, vol. 13, no. 5, pp. a467–a473, May 2025.
- [7] A. Shaik, S. S. Dutta, I. M. Sawant, S. Kumar, A. Balasundaram, and K. De, “An attention based hybrid approach using CNN and BiLSTM for improved skin lesion classification,” *Scientific Reports*, vol. 15, art. no. 15680, May 2025, doi: 10.1038/s41598-025-00025-2.
- [8] G. N. Alwakid, S. Tehsin, M. Humayun, A. Farooq, I. Alrashdi, and A. Alsirhani, “Graph Attention Networks for Skin Lesion Classification with CNN-Driven Node Features,” *Computers, Materials & Continua*, vol. 74, no. 2, pp. X–Y, Oct. 2025, doi: 10.32604/cmc.2025.069162.